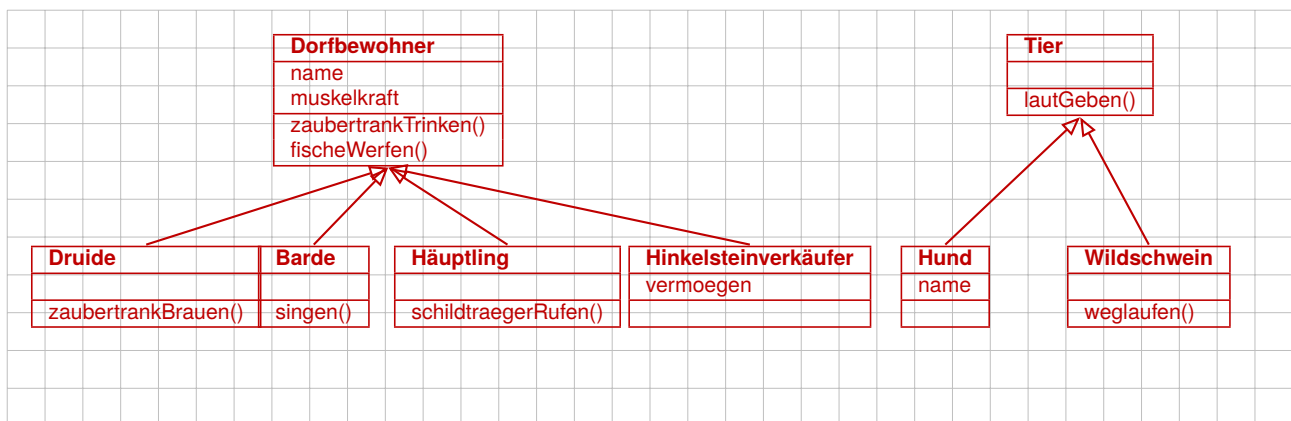


2.0.2 Vererbung: Wiederholung und Vertiefung

Aufgabe 0 Das gallische Dorf

In einem gallischen Dorf leben Druiden, Barden, Hinkelsteinverkäufer und ein Häuptling. Alle Dorfbewohner haben einen Namen und Muskelkraft, die sie durch das Trinken eines Zaubertranks vervielfachen können. Außerdem können sie Fische werfen. Nur Druiden können einen Zaubertrank brauen, Barden können singen und nur ein Häuptling kann seine Schildträger rufen. Hinkelsteinverkäufer haben hingegen ein Vermögen. Außerdem gibt es verschiedene Tiere, die Laute von sich geben können. Hunde haben zusätzlich einen Namen, während Wildschweine weglaufen können.

a) Zeichne ein Klassendiagramm, das diese Situation modelliert. **b)** Programmieren: Website.



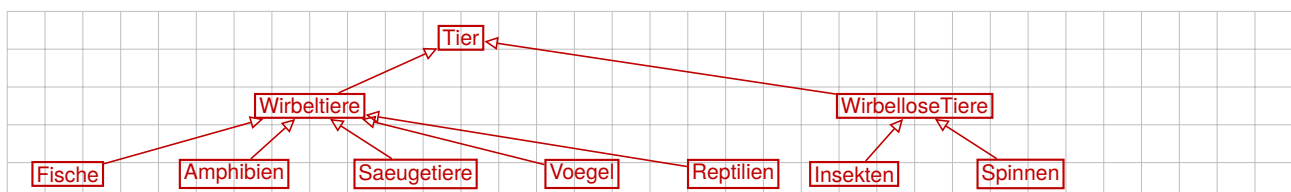
Fehlermeldung beim ersten `new Barde(...)` und ihre Lösung:

Barde hat keinen passenden Konstruktor (Konstruktoren werden nicht vererbt). Lösung: eigener

Konstruktor `Barde(String nameNeu, int muskelkraftNeu) { super(nameNeu, muskelkraftNeu);`

Aufgabe 1

Zeichne ein vereinfachtes Klassendiagramm (nur Klassennamen) zu: Tier, Wirbeltiere, WirbelloseTiere, Fische, Amphibien, Saeugetiere, Voegel, Reptilien, Insekten, Spinnen.



Vererbungshierarchie, Ober- und Unterklassen

Bei der Vererbung gibt es Unter- und Oberklassen. Eine Unterklasse besitzt automatisch alle Attribute und Methoden der Oberklasse. Es werden zwei

Richtungen unterschieden: Wird eine Unterklasse erstellt, so spricht man von einer

Spezialisierung der Oberklasse. Umgekehrt ist die Oberklasse eine

Generalisierung der Unterklasse.

Ist eine Unterklasse gleichzeitig Oberklasse einer weiteren Klasse, spricht man von **mehrstufiger**

Vererbung; es entsteht eine Vererbungshierarchie.

Vererbung und Überschreiben

Konstruktoren werden nicht vererbt. Die Unterklasse ruft den Konstruktor der Oberklasse mit super(...) auf.

Definiert man eine Methode in der Unterklasse erneut (gleicher Name, gleiche Parameter, gleicher Rückgabotyp), so wird die ursprüngliche Methode überschrieben. Sie hat in der Unterklasse eine **andere** Funktionsweise.

Die Annotation @Override vor der Methode sagt dem Compiler: Diese Methode überschreibt eine gleichnamige Methode der Oberklasse. Gibt es diese nicht (z. B. Tippfehler), meldet der Compiler einen Fehler. Ohne @Override entsteht stattdessen stillschweigend eine neue, zusätzliche Methode.

Aufgabe 2 Projekt Aquarium

a) Nenne die Attribute (mit Datentyp), die die Klasse Fisch bereitstellt.

String art, boolean tarnung (Startwert false), int geschwindigkeit

b) Beschreibe den Konstruktor von Fisch: Parameter und Wirkung.

Fisch(String art, int x, int y): setzt Position und Bild mit super(x, y, ...), speichert die Art, wählt eine zufällige Geschwindigkeit 1–4 und vergrößert das Bild.

c)–e) auf der Website. Attribute deiner Klasse Hai mit Datentyp und Startwert:

int alter = 1; boolean hungrig = true;

Aufgabe 3 Fehlersuche

Im folgenden Code haben sich Fehler eingeschlichen. Markiere sie im Code, gib in der Tabelle die Art des Fehlers an (Syntax, Datentyp, Logik) und schreibe die Korrektur daneben.

```

1 class Katze
2 {
3     String name;
4     String alter;
5     boolean geschlecht;
6
7     public Katze(String nameN, int alterN)
8     {
9         name = "Minka";
10        alter = alterN
11        geschlecht = "weiblich";
12    }
13 }

```

Zeile	Art	Korrektur
4	Datentyp	int alter;
9	Logik	name = nameN;
10	Syntax	; am Zeilenende
11	Datentyp	geschlecht = false;